



COLLEGE CODE: 9528

**COLLEGE NAME: SCAD COLLEGE OF ENGINEERING
AND TECHNOLOGY**

**DEPARTMENT : COMPUTER SCIENCE AND
ENGINEERING**

STUDENT NM ID: 598738F061328D19BE64CCCC7475B621

Roll no : 952823104070

DATE : 24.10.2025

Completed the project named as:

Phase 5

TECHNOLOGY PROJECT NAME :

Dynamic Image Slider

SUBMITTED By,

NAME: KABILAN K

Phase 5 – Project Demonstration & Documentation

1. Final Demo Walkthrough

The **Final Demo** is a live or recorded presentation that showcases the working of the project from start to finish.

- **Purpose:** To demonstrate how the system/application works in real-time.
- **Content:**
 - Brief introduction about the project title, objective, and technologies used.
 - Step-by-step demonstration of major features and functionalities.
 - Highlight the uniqueness or innovations in your project.
 - Show how real data or inputs are processed to produce meaningful outputs.
 - Conclude with performance, results, and key takeaways.
- **Tools:** You can use screen recording tools (like OBS, Loom, or Zoom) or live demo during presentation.
- **Tip:** Always keep a backup video or screenshots in case of internet or system issues.

2. Core Features Implementation

Project Report

The **Project Report** is a comprehensive document that explains every stage of your project, from idea to implementation. It serves as official documentation for academic or professional submission.

Typical structure of a Project Report:

1. **Title Page & Certificate**
2. **Abstract:** Short summary (100–150 words) of your project idea, purpose, and results.
3. **Introduction:** Problem statement, motivation, and scope of the project.
4. **Literature Review:** Related works or background studies.
5. **System Design:** Architecture diagram, data flow diagrams, ER diagrams (if applicable).
6. **Implementation:** Explanation of modules, programming languages, and tools used.
7. **Results & Discussion:** Screenshots, outputs, performance graphs, or analysis.
8. **Conclusion & Future Scope:** Summary of achievements and possible improvements.
9. **References:** Websites, research papers, or datasets used.

The report should be typed neatly in **Times New Roman, 12pt font**, with **1.5 line spacing**, and properly sectioned.

3. Screenshots / API Documentation

Screenshots:

Include clear images of your project's user interface, major modules, and output results. Screenshots help evaluators visualize your system even without running it.

- Capture login page, dashboard, output graphs, reports, etc.
- Label each image properly (e.g., *Figure 3.2: Sales Trend Graph*).

API Documentation (if applicable):

If your project involves backend APIs or web services, document them clearly using a standard format like Swagger or Postman Collection. Include:

- API endpoint URL (e.g., /api/users)
- Request method (GET, POST, PUT, DELETE)
- Request/Response structure with examples:

- Authentication or header requirements

This makes your project developer-friendly and ready for real-world integration.

4. Challenges & Solutions

During the development of this project, several challenges were encountered at different stages. Each challenge required specific problem-solving techniques and research to arrive at an effective solution.

One of the major challenges was data cleaning and preprocessing. The dataset contained several missing values, duplicate entries, and inconsistent formats which affected the accuracy of the analysis. To overcome this, data cleaning techniques such as handling null values using the `fillna()` method and removing duplicates using `drop_duplicates()` were applied. Additionally, data types were standardized to ensure smooth execution of analytical operations.

Another challenge was API integration and backend connectivity. During testing, the API responses were sometimes inconsistent due to differences in JSON structures or network delays. This created difficulties in displaying real-time data. The issue was solved by implementing proper error-handling mechanisms, schema validation, and retry logic. This ensured the API could handle unexpected responses gracefully without crashing the application.

The deployment process also posed a significant challenge. Initially, the application failed to run properly on deployment platforms like Render or Heroku due to version mismatches and missing dependencies. After carefully analyzing the error logs, environment variables were correctly configured, and the required libraries were explicitly mentioned in the requirements file. This made the project deployment smooth and stable.

Another difficulty faced was performance optimization. When working with large datasets, the execution time of certain functions was very high. To improve efficiency, optimized algorithms and vectorized operations using Pandas and NumPy were implemented. Unnecessary loops were avoided, and memory usage was minimized by selecting only the required columns during analysis.

Next, **performance testing** ensures the slider loads quickly and runs smoothly without lag or delays. Techniques such as lazy loading and optimized animations must be validated under real conditions. The development team uses tools like **Google Lighthouse** or **WebPageTest** to measure load times, memory usage, and rendering speed. The slider

5. GitHub README & Setup Guide

❑ **Project Title and Description:**

This section contains the name of the project and a brief explanation of what the project is about. It helps readers quickly understand the main idea or objective of the work.

❑ **Features and Use Cases:**

This part highlights the main functionalities or special features of the project. It can also include examples of how the project can be used in real-world scenarios.

❑ **Tech Stack or Tools Used:**

Here, the programming languages, frameworks, and libraries used in the project are mentioned (for example, Python, Flask, Pandas, React, or MySQL). This gives a technical overview of the environment in which the project was built.

❑ **Installation and Setup Instructions:**

This is one of the most important sections of the README. It provides step-by-step guidance on how to download, install, and run the project on another system. It usually includes commands for cloning the repository, installing required dependencies, and executing the main application file.

❑ **Screenshots or Demo Links:**

Visual representations like screenshots or video demo links are often added to make the README more interactive. They help users quickly visualize how the project looks and functions.

❑ **API or Dataset Information (if applicable):**

For projects that use APIs or datasets, details about their usage, endpoints, and structure are included. This ensures transparency and helps others reproduce the same results.

❑ **Contributors and Acknowledgments:**

If the project is done by a team, the names and GitHub profiles of contributors are mentioned. Acknowledgments can also be given to mentors, teachers, or external resources that supported the project.

❑ **License and Usage Terms:**

Many open-source projects include a license (like MIT or Apache) to define how others can use or modify the project. This ensures proper usage rights and credit to the original developer.

6. Final Submission (Repo + Deployed Link)

GitHub Repository (Repo):

A GitHub repository serves as the central storage location for all the project files, code, and documentation. It acts as proof of development and enables version control, collaboration, and public access to the source code. The repository should be well-organized and contain all necessary resources that allow others to understand and run the project easily.

A complete project repository must include:

- Source Code: All the main program files, scripts, and resources used in the project.
- Documentation: The README file, setup guide, and project report should be uploaded for reference.
- Screenshots / Demo Folder: A collection of images showing the application's interface and outputs.
- Requirements File: A list of dependencies or libraries (like requirements.txt in Python) needed to run the project.
- License (optional): Specifies the terms under which others can use or modify the project.

The repository link is usually shared with evaluators for code review, grading, or portfolio purposes.

